

V-model with fuzzy quality function deployments for mobile application development

Chong Hon Lim  | Jeng Feng Chin 

School of Mechanical Engineering, Engineering Campus, Universiti Sains Malaysia, Penang, Malaysia

Correspondence

Jeng Feng Chin, School of Mechanical Engineering, Engineering Campus, Universiti Sains Malaysia, 14300 Nibong Tebal, Penang, Malaysia.

Email: chinjengfeng@usm.my

Funding information

This work was supported by Research University Grant (RUI), Universiti Sains Malaysia (grant number 8014069).

Abstract

Mobile application (app) development is a subset of software development distinguished by a short development life cycle, rapidly changing business requirements, mobile device technical specifications, and operating system compatibility. The study presents a V-model mobile app development technique that combines two fuzzy quality function deployment (FQFD) phases. In mobile app development, user requirements are often misunderstood, resulting in poor mobile app design. The research motivation for employing FQFD is to obtain more accurate deductions of human (user or design team) inputs affected by intrinsic vagueness and ambiguity in linguistics. The combination hence delivers synergistic benefits, where FQFDs more effectively solicits and breaks down the voice of the customer (VOC) and system requirements, and the V-model aligns the development to the broken-down requirements. The development of an online statistical process control app was demonstrated. FQFD structurally related user requirements, system requirements, and design strategies in the V-model's verification phases. The validation phases used test plans created during the verification phases to detect and correct programming errors. In the final validation stage, the focus group agreed that the mobile app met all of the user requirements. This study adds to the methodology's novelty by proposing and demonstrating the integration of the FQFD and V-model into the development of mobile apps. As a result of the limitation, other areas of software engineering, such as time and resource coordination and release management, are not considered.

KEYWORDS

fuzzy quality function deployment, mobile application development, software development, V-model

1 | INTRODUCTION

Mobile application (app) development is a subset of software development distinguished by a short development life cycle, rapidly changing business requirements, mobile device technical specifications, and operating system compatibility.¹ In the literature, numerous mobile app development methodologies have been offered, such as waterfall, spiral, V-model, incremental development, concurrent development, component-based software development, prototype techniques, and agile methods.^{2–4} With the proliferation of mobile devices and apps, new methodologies are constantly introduced. These methods have demonstrated with different strengths and weaknesses. For example, agile methods have been further developed into extreme programming (XP), SCRUM, Lean Development, and dynamic systems development. Agile methods, according to Al-Ratrout,³ are simple adaptation to change, support for collaboration between developers and clients, reduction in delivery time, and simplification of the development process. However, agile method limitations necessitate active user iterations throughout development, accompanied by

insufficient and unclear requirements, and difficulty in integration testing due to the iterative and incremental development approach.⁵ Additionally, agile methods rely on people and their creativity rather than on processes for overcoming the problems of traditional software methodologies.⁶ By defining structure and alignment in software development, the V-model is frequently more advantageous than agile methods under these limitations.

V-model, proposed by Brühl in 1995, has gained popularity in recent decades.⁷ The V-model divides development into sequential and matching verification and validation phases to improve productivity and the early intervention of developmental errors. As illustrated in Figure 4 (the original V-model), the verification activities fall on the left side of the model while the validation activities are on the right.^{8,9} The verification and validation are equivalent to decomposition and recomposition,¹⁰ where the former technically breaks down the program needs into smaller tractable parts of code and the latter assembles them into the final product. The verification starts with the requirement analysis phase to define users' needs and requirements. An acceptance test design is conceived to later validate these requirements. The next phase deals with system design, and the role includes defining the system architecture to produce an overview of the relevant solution, platform, system, product, and service/process. A system test design is also created to test how the individual modules work together as a system. Phase 3 relates to architecture design and aims to identify the list of modules and their specifications in supporting the system. The integration test design is carried out in this phase. Phase 4 performs module design to define each module's logic or programming function. The unit test design is developed. The coding phase resides at the bottom of the V-model, where the module designs are programmed per specifications. The development moves into the validation phases at the end of the coding phase. The unit testing phase debugs individual module coding at the early programming stages, followed by the integration testing phase to test the interfacing between different components in a system. In the system testing phase, the system is tested as a whole. Finally, the acceptance testing phase, normally performed from a user application point of view, validates if the software meets design requirements. Iteratively, when problems are encountered during a test phase, the model reverts to the corresponding verification phase for corrective action.¹¹ The V-model is suitable for small projects because of its simplicity and easy-to-understand requirements.¹² The V-model has been widely used in software development, such as safety-critical systems,¹³ rental car applications,¹⁴ and driving distance estimation software for electric vehicles.¹⁵ In contrast, adoption of the V-model in mobile app development is scarce, as evidenced by a scan of the relevant literature yielding few examples.^{16–18}

The V-model's verification and validation phases have been subjected to various enhancements in literature. Han et al.⁶ presented a value-driven V-model for verification phases by adding an inner process to the V-model to elicit customer values during the requirement analysis phase and drive test designs throughout the validation stages. Greassler¹⁶ incorporated into the V-model, continuous requirements' elicitation and management, comprehensive modeling and analysis on overcoming gaps and fractures between models, and focus on sales and marketing feedback to gain impulses for product innovations. After product delivery, Mathur and Malik¹⁹ introduced software and maintenance testing into the V-model's validation phases for software testing management strategy. Durmus et al.²⁰ enhanced V-model by adding a discrete event system (DES)-based fault diagnosis method as an intermediate step between module design and coding phase for software development processes of safety instrumented system (SIS). Essebaa and Chantit¹⁴ integrated model-based testing in V-model to manage testing in evolutionary software systems.

In verification phases, the decomposition criteria are frequently misunderstood in software engineering. The reason is the contemporary V-model does not depict the process of collecting and eliciting user requirements (URs).²¹ The implications are losing focus on what customers deem valuable and poor customer satisfaction.⁶ According to Dasanayake et al.,²² requirements define software architecture, and any suboptimal decision in this definition process can degrade system quality and cause problems later in the development process. On this premise, the study proposes a V-model integrating fuzzy quality function deployment (FQFD) for mobile app development. Quality function deployment (QFD) is a methodology for gathering the voice of the customer (VOC) and translating it into URs. Given that inherent vagueness and uncertainty in human linguistics affect user solicitation, fuzzy set theory (hence FQFD) is introduced to obtain more accurate deductions of URs.²³ Although there are related studies combining QFD and V-model,^{24,25} no research has integrated FQFD and V-model in software or mobile app development. As a result, the research objective is to demonstrate the suitability of V-model to use multi-FQFD as the front end to elicit and decompose URs and then the system requirements (SRs). The decomposed elements are then used to help design test plans, and the identified priorities can be used to determine the project's focus. Unlike Han et al.,⁶ the deployment of FQFD does not necessitate the use of a complex method such as the analytic hierarchy process (AHP) to determine the weight of customer requirements. The remainder of the V-model would ensure systematic fulfillment, programming, and validation of app specifications.

The work begins with developing a suitable model and then demonstrating the viability of integration through a case study. The model enabled a rigorous understanding of customer requirements, explicit alignment in the design of suitable features to different levels of coding, and focused app development control. A manufacturing case study is purposefully chosen because mobile devices play an increasingly crucial role in manufacturing digitization and cyber-physical systems.^{26,27} The research contribution is an improved V-model towards user-centered design through integrating FQFDs to elicit URs that are often vague and misunderstood. With largely common steps, the model can be generic and adaptable to other development-based projects. However, the development has only reached the proof-of-concept stage. Managerial aspects of software engineering, such as team selection and formation, time and resource coordination, and release management, are also beyond the focus of the model. Nonetheless, these managerial aspects do not contradict the model and can be used in practical applications.

The study is structured as follows. Section 2 presents the literature review. Section 3 introduces the research methodology of the proposed model. Section 4 illustrates the mobile app development based on a case study. Section 5 discusses the findings before the conclusion.

2 | FQFD

In the 1960s, Yoji Akao introduced QFD to Japanese industries.²⁸ QFD ensures that product development processes and features are aligned and focused around the VOC.^{29,30} The house of quality (HOQ) is the fundamental structure of QFD (Figure 1). It comprises WHATs, HOWs, a relationship matrix, a roof matrix, and importance ratings (WHATs and HOWs). While most organizations use a single QFD,³¹ multiple QFDs are frequently used for complicated projects or to evaluate VOCs.^{32,33}

Fuzzy numbers are used as linguistic inputs in FQFD. Linguistic inputs appear intuitive and reasonable to avoid the subjective and arbitrary quantification of input data common in human decision-making.³⁴ A fuzzy number is derived from fuzzy set theory and corresponds to a value between [0, 1], which can be represented mathematically or graphically, as shown in Figure 2. While 0 and 1 represent the minimum and maximum degrees of membership, respectively, the intermediate values represent “partial” membership.³⁴

Triangular fuzzy numbers (TFNs) are the simplest and most common type of fuzzy numbers.^{23,35–37} Each TFN represents a linguistic variable and forms a triangle of the membership function $A = (a_\alpha, a_\beta, a_\gamma)$, representing the fuzzy number's lower limit, closest fit, and upper limit.³⁸ The most common type of TFN in FQFD is symmetrical TFN. TFNs can be subjected to various arithmetic operations following the rules outlined in Table 1.

A predetermined set of fuzzy numbers (points) spreads evenly across the scale of 1 to 10 in a fuzzy number system (Figure 3(A)). Miller defines seven plus or minus two points as the most information an observer can provide about the objects based on an absolute judgment.⁴⁰ When gauging survey respondent preferences, 7-point, 9-point, and 10-point scales express respondents' feelings adequately and are thus preferred over 5-point or fewer scales.^{41,42} A 5-point linguistics scale provides enough information to measure the strength of the relationship with minimal uncertainty.³⁸ A 5-point scale (Figure 3(B)) with symmetrical TFNs (0,1,2), (2,3,4), (4,5,6), (6,7,8), and (8,9,10) is simple and should be used whenever possible, according to Chan and Wu.³⁶ Intermediate points, such as TFNs (1,2,3), (3,4,5), (5,6,7), and (7,8,9), can be introduced when a compromise must be made or uncertainty exists.

Following an introduction to fuzzy numbers and their systems, the implementation of FQFD is discussed. FQFD can be implemented in six steps, {S1, S2, S3, S4, S5, S6}, according to Zahedi et al.³⁷ S1 identifies the WHATs by eliciting URs in their own words in direct discussion or interviews, surveys, focus groups, customer specifications, and observation.³¹ Assume M WHATs have been identified and are represented as follows:

$$\text{WHATs} = \{w_m, \text{ where } m = 1, 2, \dots, M\}$$

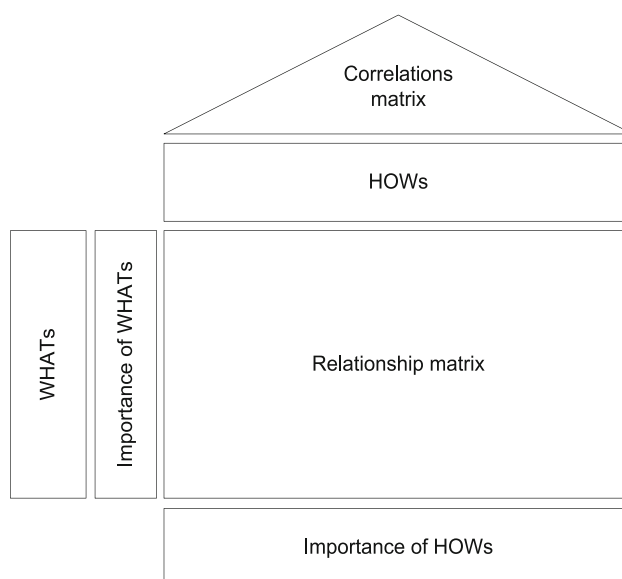


FIGURE 1 House of quality (HOQ)

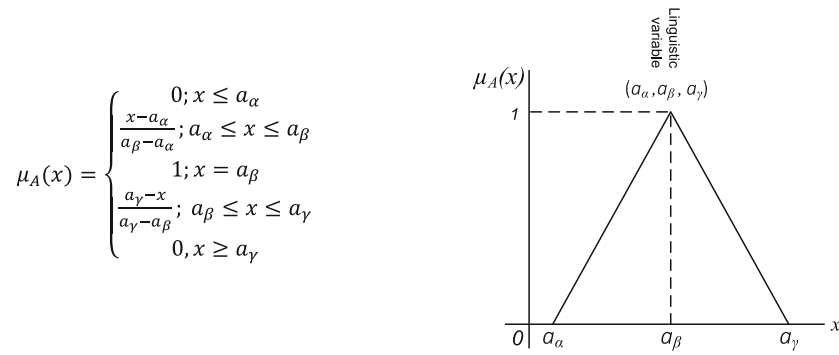


FIGURE 2 Triangular fuzzy number (TFN) in the mathematical and graph forms

TABLE 1 Arithmetic operations of triangular fuzzy number (TFN)³⁹

Arithmetic operation	Notation	Rule
Addition	$A \oplus B$	$(a_\alpha + b_\alpha, a_\beta + b_\beta, a_\gamma + b_\gamma)$
Subtraction	$A \ominus B$	$(a_\alpha - b_\gamma, a_\beta - b_\beta, a_\gamma - b_\alpha)$
Scalar multiplication	$d \otimes A$	$(da_\alpha, da_\beta, da_\gamma)$
Multiplication	$A \otimes B$	$[\min(a_\alpha b_\alpha, a_\alpha b_\gamma, a_\gamma b_\alpha, a_\gamma b_\gamma), a_\beta b_\beta, \max(a_\alpha b_\alpha, a_\alpha b_\gamma, a_\gamma b_\alpha, a_\gamma b_\gamma)]$
Division	$A \oslash B$	$[\min(a_\alpha/b_\alpha, a_\alpha/b_\gamma, a_\gamma/b_\alpha, a_\gamma/b_\gamma), a_\beta/b_\beta, \max(a_\alpha/b_\alpha, a_\alpha/b_\gamma, a_\gamma/b_\alpha, a_\gamma/b_\gamma)]$

Note: A and B are both TFNs, where $A = (a_\alpha, a_\beta, a_\gamma)$ and $B = (b_\alpha, b_\beta, b_\gamma)$. d is a scalar value, where $d > 0$.

S2 uses linguistic variables to rank the importance of WHATs. Assume K users are enlisted to complete the rating, their individual feedback was first transformed into TFNs. A TFN, w_{mk} , is defined by a triplet $(w_{mk\alpha}, w_{mk\beta}, w_{mk\gamma})$. Then, the importance mean of w_m can be computed based on Equation (1).

$$w_m = \frac{1}{K} \otimes (w_{m1} \oplus w_{m2} \oplus w_{m3} \oplus \cdots \oplus w_{mK}) \quad (1)$$

The HOWs, or technical attributes, are determined by S3. Assume N HOWs have been discovered and are labeled as follows:

$$\text{HOWs} = \{h_n, \text{ where } n = 1, 2, \dots, N\}$$

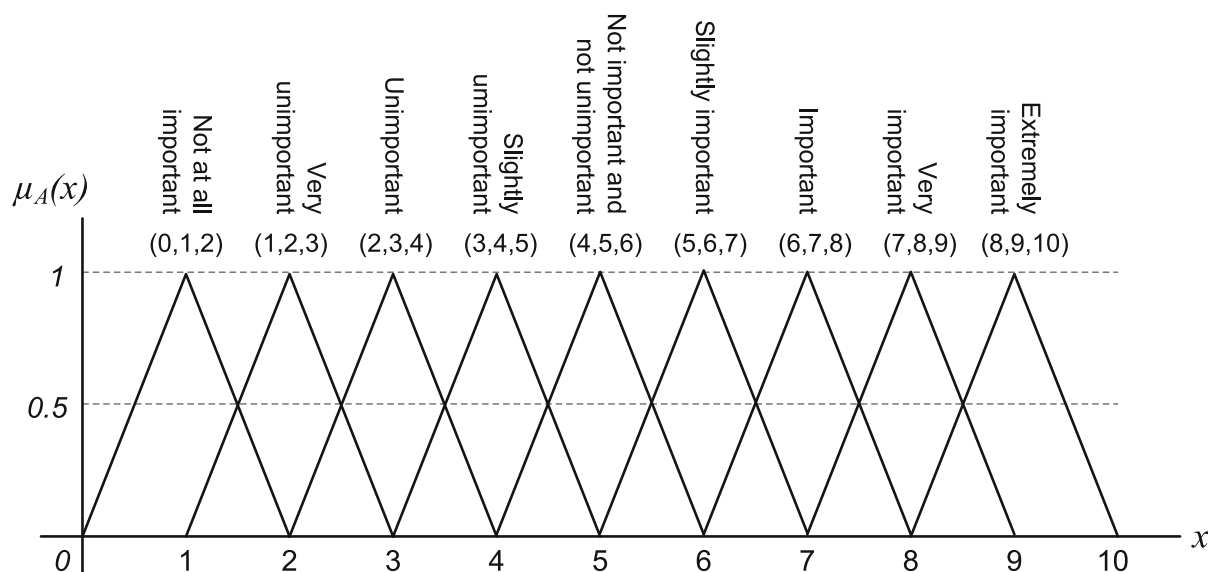
In S4's relationship matrix, WHATs are the rows, and HOWs are the columns. The team members assess the impact of each HOW on each WHAT and assign a linguistic strength indicator in the relationship matrix. s_{nm} is the triplet TFN of the relationship score between the n th HOW and the m th WHAT, $s_{nm} = (s_{nm\alpha}, s_{nm\beta}, s_{nm\gamma})$. According to Equation (2), S5 determines the relative importance of the HOWs in the form of TFNs, $r_n = (r_{n\alpha}, r_{n\beta}, r_{n\gamma})$.

$$r_n = \frac{1}{M} \otimes [(s_{n1} \otimes w_1) \oplus (s_{n2} \otimes w_2) \oplus \cdots \oplus (s_{nM} \otimes w_M)] \quad (2)$$

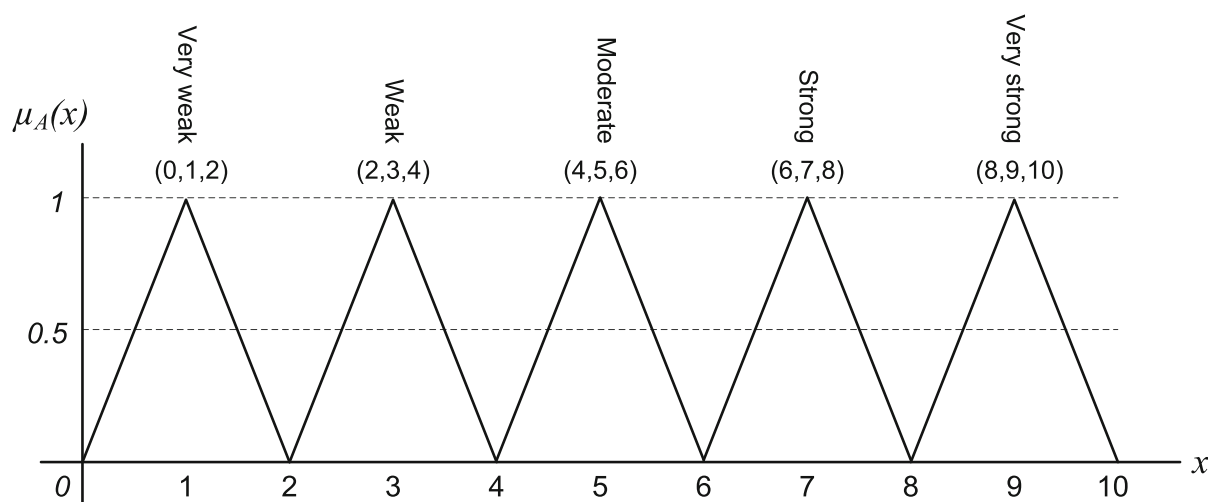
The relative importance rating is defuzzified with Equation (3) to rank the HOWs⁴³:

$$\bar{r}_n = \frac{r_{n\alpha} + 2r_{n\beta} + r_{n\gamma}}{4}. \quad (3)$$

Finally, Equation (4) determines the relative importance (in percentage) of each HOW (4). Relative significance = Z_n , where $n = 1, 2, \dots, N$.



(A) A 9-point scale with importance-based linguistic variables



(B) A 5-point scale with relation-based linguistic variables

FIGURE 3 Different scales and linguistic variables in fuzzy number system

$$Z_n = \left[\bar{r}_n / \sum_{i=1}^N \bar{r}_i \right] \times 100. \quad (4)$$

S6 expands on the correlations matrix by looking at which HOWs are correlated and how strong the correlation is. Meanwhile, an action plan is developed to refine critical HOWs and prioritize development tasks and resources.³¹

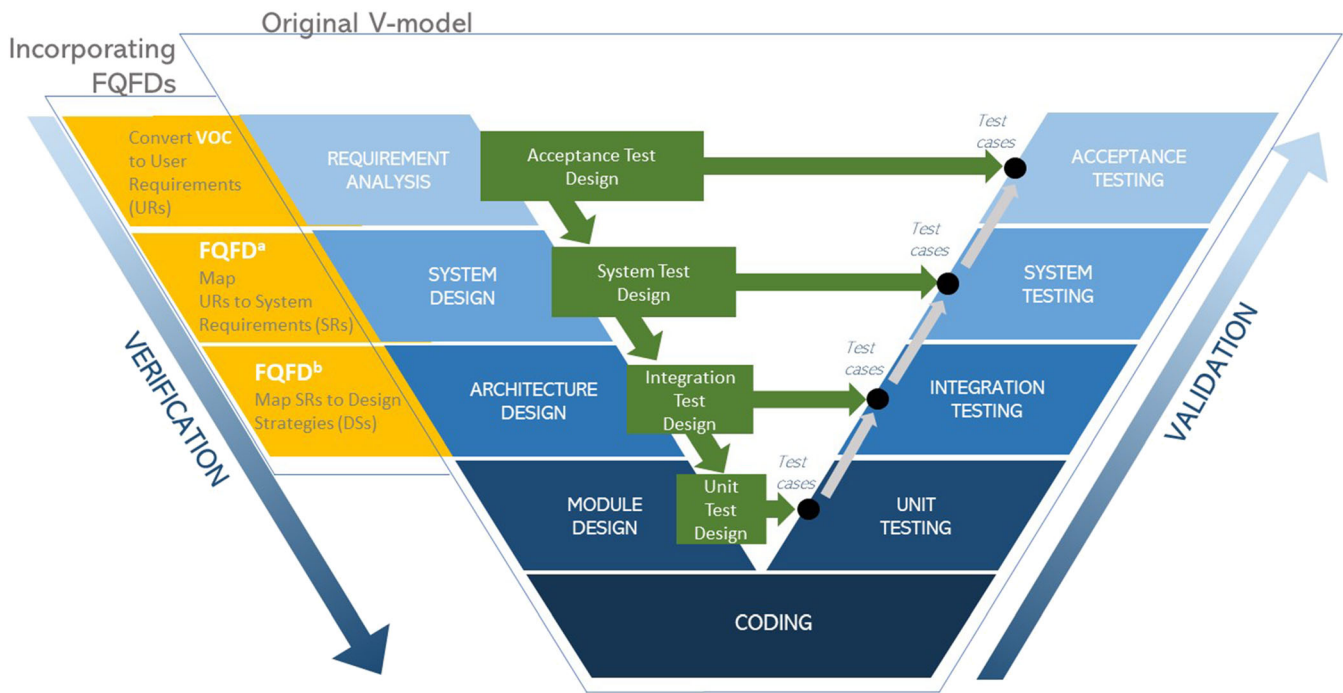


FIGURE 4 The V-model with FQFDs. FQFD, fuzzy quality function deployment; VOC, voice of the customer

3 | V-MODEL WITH FQFDs

The study proposes a V-model integrating with FQFDs for app development, using two FQFDs as the front end to solicit and decompose customer requirements and then mapping to the design requirements. The decomposed elements can then be used to help design test plans, and the identified priorities can be used to determine the project's focus. FQFDs aid domain analysis by determining which features are part of the app and how they are supported based on VOC. Verification employs two FQFDs, denoted by FQFD^a and FQFD^b. Their HOQs are thus denoted as HOQ^a and HOQ^b, with constituent steps S1^a, S2^a, and S3^a; S4^a, S5^a, and S6^a; and S1^b, S2^b, S3^b, S4^b, S5^b, and S6^b. FQFD^a is used in the requirement analysis and system design phases, whereas FQFD^b is used in the architecture design phase. Figure 4 depicts the improved V-model. Each phase is explained in detail below.

3.1 | Requirement analysis phase

This phase is concerned with developing the ideal SRs, with the URs of the proposed system being obtained through the analysis of commonly collected VOC via survey. However, it does not dictate how the software will be conceived or built. S1^a and S2^a of FQFD^a's six steps are used here to determine URs, which are indicative of the VOC. In S1^a, a list of expected URs ("WHATs") are conceived based on potential app features,⁴⁴ of which are defined as "the prominent or distinctive user-visible aspects, quality, or characteristic" (of the app).⁴⁴ These requirements are typically expressed in terms of expected system operations, physical elements, interface, performance, data, and security. In S2^a, potential users are asked to rate the relative importance of each UR using an appropriate instrument. During the system design phase, the URs and their importance would act as a guide for the system designers. An acceptance test is also designed to validate the URs.

3.2 | System design phase

URs are analyzed and understood in the context of the proposed app during the system design phase. The phase determines the methods and strategies for implementing the URs. The final four steps of FQFD^a are used to identify and link URs and SRs in HOQ. In HOQ^a, S3^a proposes a series of SR ("HOWs"). S4^a connects URs to SRs, demonstrating how the proposed attributes meet users' expectations. The relationship strengths indicate how well an SR meets the URs. S5^a calculates the relative importance ratings using symmetrical TFN and ranks all SRs to meet overall

URs. $S6^a$, located on the HOQ's "roof," determines the correlations between individual SRs to plan the implementation strategy. Furthermore, system testing is intended to validate the SRs and examine overall system functionality under various use cases.

3.3 | Architecture design phase

The next phase determines system architecture design (high-level design) to produce an overview of the relevant solution, platform, system, product, and service/process. This phase also prioritizes strategies to fulfill the SRs. In FQFD^b, the SRs and their importance (in TFN) are now transferred as WHATs, as in $S1^b$ and $S2^b$. The HOWs are redefined in $S3^b$ as the system design strategy (DS). $S4^b$ determines the relationship strength between the DSs and SRs. The result of $S5^b$ underlines the relative importance rating of all DSs in fulfilling the overall SRs. The ranking of the DSs also ensures adequate allocation of design attention in the downstream development process. Finally, $S6^b$ develops the matrix of correlations between DSs to decide which DSs are best implemented together or concurrently. The focus of aligning system architecture design to DSs is to identify the system architecture, modules, and required tasks mapping to the DS. App specification is created to act as a template for the development process. It can be given in various formats, including entity diagrams and data dictionaries. In parallel, integration test design is developed to the correct functioning between different sub-components, such as whether they exhibit unexpected behavior that does not occur when used in isolation.

3.4 | Module design phase

This phase involves module design, which defines each module's logic or programming function. The designer defines program module specifications and their implementation methods based on the tasks determined in the architecture design phase. The data structure, internal linking, routines, procedures, and execution sequence are examples of details. Finally, the unit test design is performed to validate the behavior of each module.

3.5 | Coding phase

The coding phase is located at the bottom of the V-model and entails program coding activities following module design and specifications. A short iterative cycle is a common coding approach. Each module may contain several sub-modules that each perform a specific function. In a short iterative cycle, completed sub-modules are tested in a white box and modified if errors are detected. Any violation of programming language syntax and type system and any behavioral discrepancy is defined as errors.⁴⁵

3.6 | V-model validation phases

Validation also contains four phases. At the lower level, unit testing is performed in a white box environment where the codes are visible. Unit test cases are applied for the designer to observe the written code's running, output, or behavior and make changes or eliminate any detected errors. Integration testing begins once all standalone modules' functionality has been validated. The goal is to validate program tasks that run cross-module functionality and expose flaws when these modules interact. System testing is then performed on the system to validate overall system behavior versus SRs. Given the nature of the test, integration and system testing are considered "black-box." Finally, acceptance testing determines whether or not the system meets the requirements specified during the requirements analysis phase. A focus group would test the app and then be polled on their impressions of its functionality and user experience (UX). Any failures during validation would necessitate immediate corrective action before moving on to the following phase.

4 | CASE STUDY

The app in the case study is built to perform short-run statistical process control (SPC). Product quality is critical in the manufacturing industry, particularly in the short run where the quantity to manufacture is small. In quality, the product's fitness for use is inversely proportional to its variability. As a result, quality improvement is the attempt to eliminate variation in processes and products.^{46,47} SPC typically employs Shewhart control charts and process capability indices for online process monitoring and the early detection of assignable causes and process shifts, allowing for proper investigation and corrective action. Specifically, sample data relevant to a process quality characteristic are collected in predetermined frequency and time. X-bar and R (range) charts, two common Shewhart control charts, are plotted in tandem to monitor the mean and range of

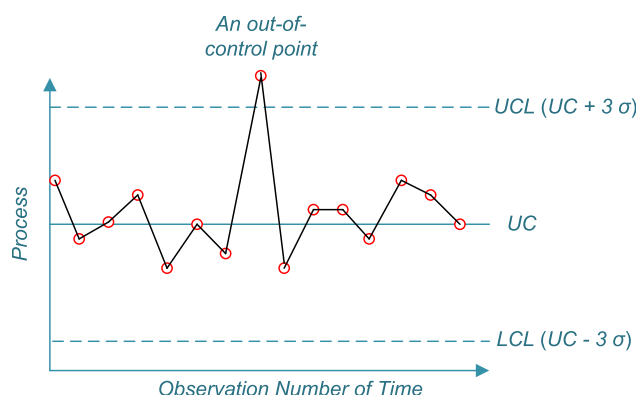


FIGURE 5 Control chart. LCL, lower control limit; UCL, upper control limit

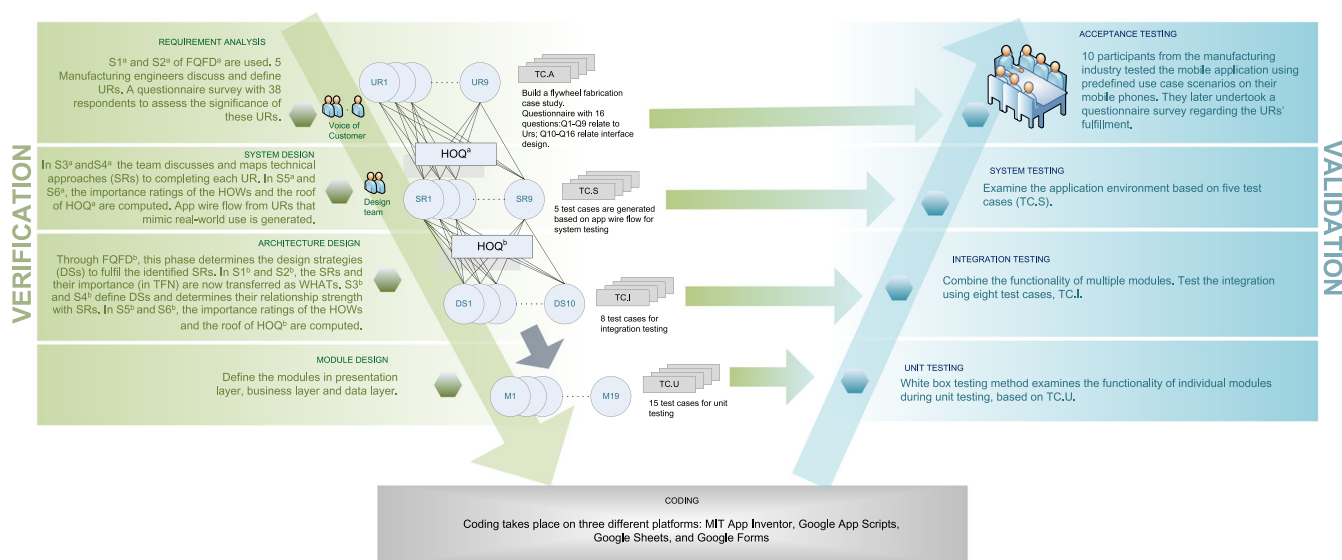


FIGURE 6 The overview of the flow of the proposed V-model. FQFD, fuzzy quality function deployment; HOQ, house of quality; SR, system requirement; UR, user requirement

TABLE 2 URs

Categories	ID	URs (WHATs)
Availability	UR1	Updated data availability
Cost	UR2	Low implementation cost
Functionality	UR3	Cater for short production runs
	UR4	Automated data processing
	UR5	Statistical process monitoring
	UR6	User alert feature
Security	UR7	Data security
UX design	UR8	User customizable
	UR9	Ease of use

Abbreviations: UR, user requirement; UX, user experience.

these data over time. In a control chart (Figure 5), a centerline (CL) represents the quality characteristic's mean value corresponding to the in-control state. Two other horizontal lines, the upper control limit (UCL) and the lower control limit (LCL), are computed, and nearly all the sample points (compiled from the sample data) will fall between them if the process is in control. Additionally, Nelson rules, graph pattern interpretation

rules, are deployed to determine whether some points are out of control in the control charts. On the other hand, process capability indices are established to quantify the extent of variation experienced by a process in relation to its specification limits. A design team of two designers is involved in the app development. The overview of the flow of the proposed V-model in the case study is depicted in Figure 6.

4.1 | Requirement analysis phase

S1^a and S2^a of FQFD^a are used in this phase. Following an informed discussion among a small group of five manufacturing engineers and a review of SPC and software requirement literature,^{48,49} nine URs are identified (Table 2).

An online questionnaire survey is then conducted to assess the significance of these URs. The survey drew 38 respondents from a pool of manufacturing engineers or professionals. The questionnaire comprises nine closed-ended questions based on the URs, with 9-point linguistic inputs (Figure 3(A)). The survey responses are converted to TFNs, and the mean of w_i importance is calculated using Equation (1). In parallel, an acceptance test in the form of a “usability test” is designed to validate VOC fulfillment. The questionnaire contains 16 questions, the first nine derived from the UR identified. Questions 10–16 focus on the user perception of the UX when using the app, covering aspects such as app responsiveness, screen refresh speed, and user-friendliness of the graphical user interface (GUI) navigation, to assess the app's ease of use (UR9) further. During the test, selected potential users from manufacturing run the app on an imaginary short-run flywheel fabrication to provide context. A flywheel is a mechanical device attached to a rotating shaft that helps to improve the power delivery. The fabrication information comes from a local machining job shop that customizes and delivers parts to the machinery and equipment industry. In the job shop, three critical-to-quality processes in the flywheel fabrication are selected, namely, precision milling, drilling, and lathe machining. Usually handled by different production personnel, these processes are run semi-automated and using computer numerical control devices. As illustrated in Figure 7,

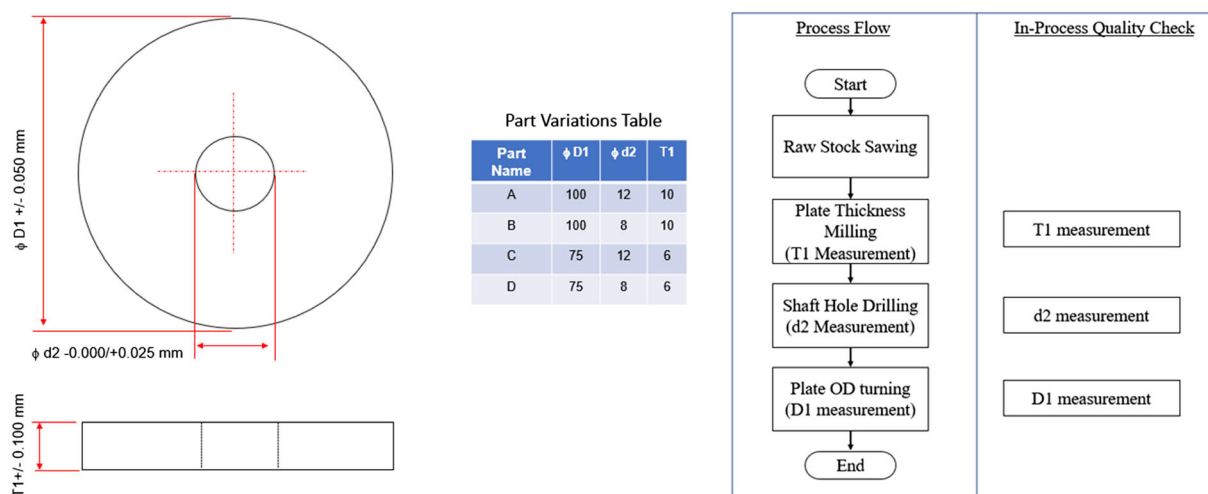


FIGURE 7 Summarized flywheel fabrication process flow and in-process quality check

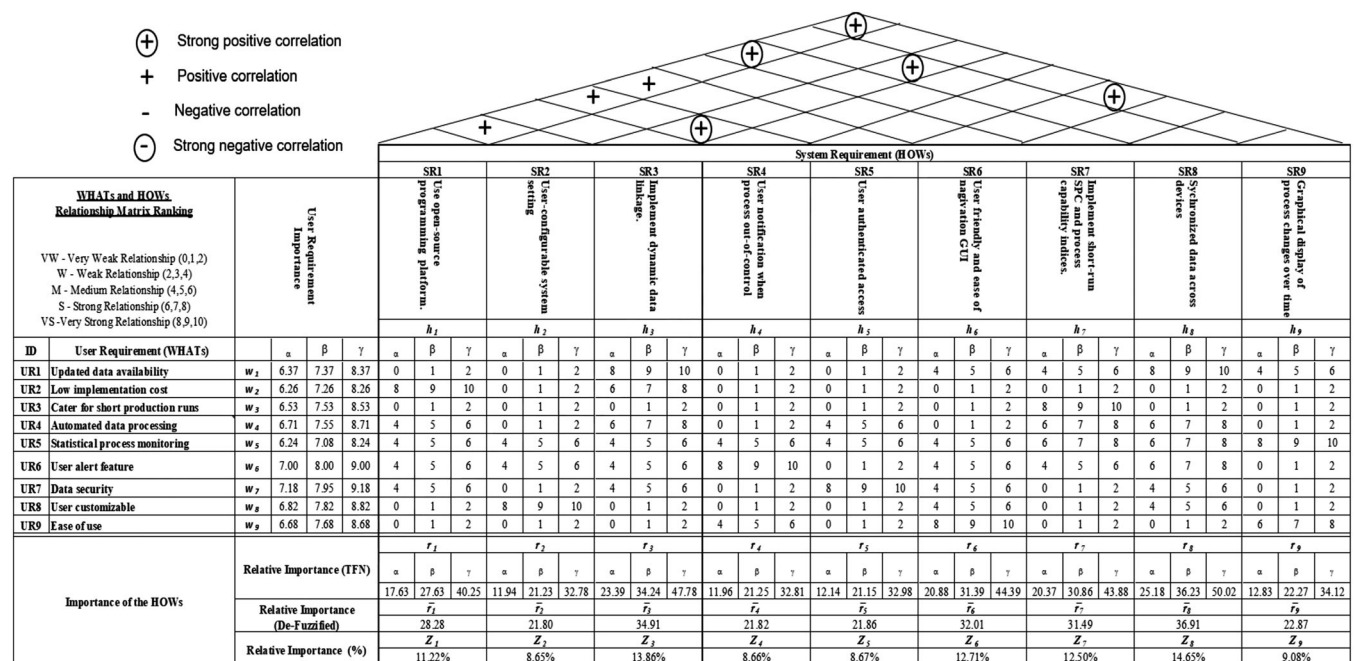
TABLE 3 SRs

ID	SR (HOWs)
SR1	Use open-source programming platform
SR2	User-configurable system setting
SR3	Implement dynamic data linkage
SR4	User notification when process out-of-control
SR5	User authenticated access
SR6	User-friendly and ease of navigation graphical user interface (GUI)
SR7	Implement short-run SPC and process capability indices
SR8	Synchronized data across devices
SR9	Graphical display of process changes over time

Abbreviations: SPC, statistical process control; SR, system requirement.

TABLE 4 User requirement (UR) and system requirement (SR) relationship matrix

	SR1	SR2	SR3	SR4	SR5	SR6	SR7	SR8	SR9
UR1	VW	VW	VS	VW	VW	M	M	VS	M
UR2	VS	VW	S	VW	VW	VW	VW	VW	VW
UR3	VW	VW	VW	VW	VW	VW	VS	VW	VW
UR4	M	VW	S	VW	M	VW	S	S	VW
UR5	M	M	M	M	M	M	S	S	VS
UR6	M	M	M	VS	VW	M	M	S	VW
UR7	M	VW	M	VW	VS	M	VW	M	VW
UR8	VW	VS	VW	VW	VW	M	VW	M	VW
UR9	VW	VW	VW	M	VW	VS	VW	VW	S

**FIGURE 8** The HOQ^a. GUI, graphical user interface; HOQ, house of quality; TFN, triangular fuzzy number; SR, system requirement; UR, user requirement

fabrication begins with singulating a circular plate (workpiece) from raw brass stock and then precision milling to trim its thickness to the desired value. Drilling creates the shaft hole with diameter d2, and lathe machining reduces the outer diameter to the required diameter, D1. After each process, a quality control inspection (in-process quality check) is performed using a coordinate measuring machine with five random samples per lot.

4.2 | System design phase

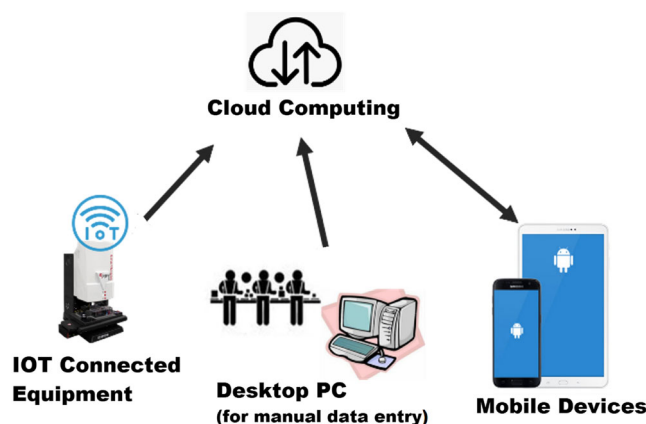
In S3^a, the team discusses technical approaches to completing each UR. All listed points are then grouped according to their respective URs and streamlined into nine SRs, as shown in Table 3.

The team investigates the interrelationships of HOWs and WHATs in S4^a. It determines the strength of the interrelationships using 5-point linguistic variables (Figure 3(B)). Table 4 summarizes the findings.

HOQ^a (Figure 8) is constructed. The importance ratings of the HOWs are computed with Equation (2) and defuzzified in S5^a with Equation (3). Equation (4) is then used to determine the relative importance of each HOW. According to the results, “synchronized data across devices” (SR8), “implemented dynamic data linkage” (SR3), and “user-friendly and ease of navigation graphical user interface (GUI)” (SR6) contribute 41.21% of

TABLE 5 Design strategies (DSs)

ID	DS (HOWs)
DS1	Data connectivity via mobile devices and internet
DS2	User configuration GUI
DS3	Leveraging open-source development tools and cloud computing platforms
DS4	User alert with Push notification
DS5	Password user login
DS6	Touchscreen operations providing natural user interface
DS7	GUI design with clarity of layout and ease of navigation
DS8	DNOM and process capability indices (PCI)
DS9	Cyclical data processing and refreshing
DS10	Shewhart control charts and Nelson rules

**FIGURE 9** Client-server architecture design

the total importance and thus have a high impact on user satisfaction. The system design phase would prioritize these three SRs over less important ones, such as “user notification when process out of control” and “user-configurable system setting.”

S6^a establishes the co-relationship between HOWs and creates the “roof matrix” for HOQ^a. “Choosing an open-source programming platform” (SR1) is critical for achieving SR2, SR3, SR5, SR7, and SR9. SR3 is advantageous to SR4 and SR8. Finally, SR6 is related to SR9 because a graphical representation of process change over time contributes to GUI design. Five test cases are designed based on actual use case scenarios to validate the completed app functionality, where the execution of each test case allows for combined validation of fulfillment to various SRs. The details of the system test are presented in the system testing phase.

4.3 | Architecture design phase

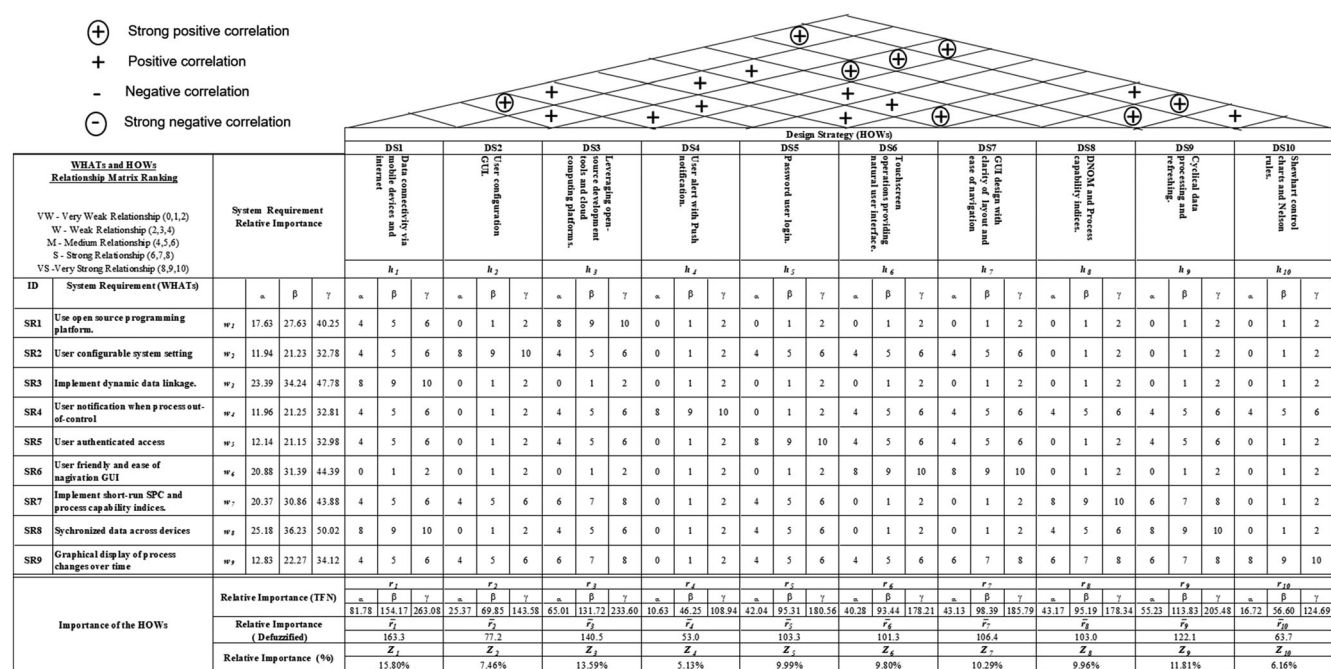
Through FQFD^b, this phase determines the DSs to fulfill the identified SRs. SRs represent WHATs in the HOQ^b in S1^b and S2^b, and their importance is rated. The team determined the DSs (HOWs) in S3^b, as shown in Table 5. A client-server architecture (Figure 9) aids in the realization of DS1. The architecture uses cloud computing, in which a cloud server handles all aspects of data storage, processing, and analysis. Thin clients are mobile devices and data entry stations (IoT-connected equipment or desktop PC for manual data entry) that communicate with the cloud server in real time via the internet. Open-source cloud-based application platforms would be considered following DS3. DS8 mandated the installation of Deviation from Nominal Method (DNOM) and process capability indices as the SPC of choice, whereas DS10 chose the X-bar and range control charts with Nelson rules to detect any variable out of control. To meet DS9 requirements, time intervals are set to refresh the control charts and Nelson rules to retrieve data in comma-separated values from any IoT-connected equipment. Following DS5, the user must be authenticated to access the app. Roles for content management are also defined. To implement DS2, the app must be configurable via a suitable GUI. When a process abnormality is detected, the app sends a message to a targeted mobile device, as required by DS4.

Table 6 shows the results of S4^b's evaluation of the relationship between WHATs and HOWs. TFNs quantify and summarize the linguistic variables in the evaluation in HOQ^b (Figure 10).

TABLE 6 System requirement (SR) and design strategy (DS) relationship matrix

	DS1	DS2	DS3	DS4	DS5	DS6	DS7	DS8	DS9	DS10
SR1	M	VW	VS	VW	VW	VW	VW	VW	VW	VW
SR2	M	VS	M	VW	M	M	M	VW	VW	VW
SR3	VS	VW	VW	VW	VW	VW	VW	VW	VW	VW
SR4	M	VW	M	VS	VW	M	M	M	M	M
SR5	M	VW	M	VW	VS	M	M	VW	M	VW
SR6	VW	VW	VW	VW	VW	VS	VS	VW	VW	VW
SR7	M	M	S	VW	M	VW	VW	VS	S	VW
SR8	VS	VW	M	VW	M	VW	VW	M	VS	VW
SR9	M	M	S	VW	M	M	S	S	S	VS

- ⊕ Strong positive correlation
 + Positive correlation
 - Negative correlation
 ⊖ Strong negative correlation



Relative importance of design strategies: DS1 > DS3 > DS9 > DS7 > DS5 > DS8 > DS2 > DS10 > DS4

FIGURE 10 HOQ^b. DS, design strategy; GUI, graphical user interface; HOQ, house of quality; TFN, triangular fuzzy number

The team determines DS's relative importance rating in S5^b. According to the findings, the top three DSs account for 41.05% of the total importance of the HOWs (DS1, DS3, and DS9). As a result, they concentrate on the architecture design phase.

The team populates the "roof matrix" by repeating S6^b. The roof matrix revealed that DS3 is essential to many DSs (DS4, DS5, DS8, DS9, and DS10). Furthermore, DS1 enables DS3, DS4, and DS9. An integration test with eight test cases is planned to investigate combined module operations and validate the assembling and integration of app modules. The test cases were created to cover the critical tasks involving the data path across the mobile app–internet–cloud server, as described in the validation sections.

4.4 | Module design phase

Modules (Table 7) are task-specific program routines defined according to the system architecture. These modules are built using open-source programming platforms, such as MIT App Inventor 2, Google App Scripts, Google Sheets, and Google Forms. The front end is the presentation layer, made up of GUI modules. As DS6 emphasizes, the GUIs display information and user feedback. The business layer is in charge of system operations, such as data transfer, updating, processing, and analysis. The data layer comprises local data storage in client devices and the manufacturing SPC process database in the cloud. As described in the unit test phase, 15 test unit test cases are devised to validate all modules,

TABLE 7 App module list

ID	Module	Architecture layer	Server/client	Description	Platform
M1	User login GUI	Presentation	Client	Authentication of identity requires the use of a username and a password.	App Inventor
M2	Dashboard GUI	Presentation	Client	The current status of all processes.	App Inventor
M3	Control chart GUI	Presentation	Client	Displaying control charts of the process of interest.	App Inventor
M4	Summary Statistics GUI	Presentation	Client	Displaying a statistical summary of the process of interest.	App Inventor
M5	App setup GUI	Presentation	Client	View and update app setup parameters.	App Inventor
M6	Data entry GUI (App)	Presentation	Client	To manually enter process data into the app.	App Inventor
M7	Data entry GUI (Web)	Presentation	Client	To manually enter process data via the workstation.	Google Forms
M8	User ID authentication	Business	Client	Authenticates the user's identity and defines the user's access authority.	App Inventor
M9	Process status indication	Business	Client	To determine the overall process status, color-coded indicators for each statistical process state are displayed in the Dashboard GUI.	App Inventor
M10	Data processing	Business	Server	In the process data spreadsheet, process, compute, and format the new data entry.	Google Sheets
M11	Alerts management	Business	Client	To manage the conditions for sending in-app notifications or text messages to registered users if low capability index or Nelson rules violations are detected.	App Inventor
M12	App setup management	Business	Server	Allow users to access and update app setup parameters in the app setup database via the app.	Google Apps Script
M13	Control charts	Business	Server	Based on the most recent n sets of process data, it generates an X-bar and a range chart.	Google Sheets
M14	Nelson rules	Business	Server	Using the eight Nelson rules, statistically evaluate the X-bar control chart data points.	Google Apps Script
M15	Auto data acquisition	Data	Server	Automatically collect CSV files from IoT-connected devices.	Google Apps Script
M16	App setup database	Data	Server	A spreadsheet for storing app configuration data.	Google Sheets
M17	Process 1 data table	Data	Server	A spreadsheet is used to store the first set of process data.	Google Sheets
M18	Process 2 data table	Data	Server	A spreadsheet is used to store the second set of process data.	Google Sheets
M19	Process 3 data table	Data	Server	A spreadsheet is used to store the third set of process data.	Google Sheets

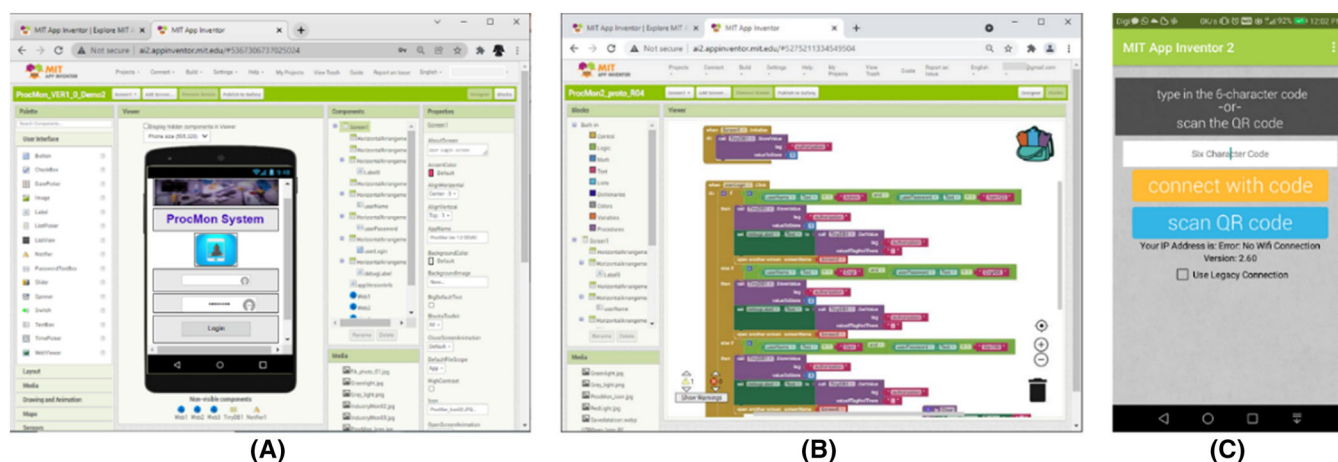


FIGURE 11 (A) App Inventor designer interface, (B) App Inventor block editor, and (C) App Inventor Companion app

TABLE 8 Unit test cases

ID	Description	ID	Description
TC.U01	User login GUI test	TC.U09	Process status indicator test
TC.U02	Dashboard GUI test	TC.U10	Data processing test
TC.U03	Control chart GUI test	TC.U11	Alerts management test
TC.U04	Summary Statistics GUI test	TC.U12	App setup management test
TC.U05	App setup GUI test	TC.U13	Control chart generation test
TC.U06	Data entry GUI (App) test	TC.U14	Nelson rules test
TC.U07	Data entry GUI (Web) test	TC.U15	Auto data acquisition test
TC.U08	User ID authentication test		

TABLE 9 Unit testing for auto data acquisition module

ID	Test step	Result
TC.U15	Run "autoCSVAcquire()" Apps script	The data in CSV files is correctly imported into "Data" sheet with correct data formatting.

debug code execution (in some cases with the option of feeding various test inputs), and compare the coded module's output behavior with the design specification.

4.5 | Coding phase

The modules are built on a variety of programming platforms. For instance, the app was created with MIT App Inventor 2. The designer interface in MIT App Inventor 2 built the app's appearance and layout, whereas the block editor built the backend logic that determines how the app works. Figure 11(A,B) depicts the creation of the app user login GUI in the designer and block editor environments. The App Inventor 2 Companion app (Figure 11(C)) is used for live app testing.

Google Apps Script implements automation scripts, such as Nelson rules, app setup management, and auto data acquisition modules. Apps Script is a JavaScript cloud scripting language that allows for the automation of tasks across Google products and the development of web applications. Finally, data processing, control chart, app setup database, and process database(s) are implemented as online spreadsheets using Google Sheets.

TABLE 10 Integration test cases

ID	Test case description	Related module(s)
TC.I01	Data entry and computation validation test (Web)	Data entry GUI (Web), data processing, Nelson rules, process database(s)
TC.I02	Data entry and computation validation test (App)	Data entry GUI (App), data processing, Nelson rules, process database(s)
TC.I03	Dashboard information display test	Dashboard GUI, process status indicator, process database(s)
TC.I04	Control charts display test	Control chart GUI, control chart generation, process database(s)
TC.I05	Statistics summary information display test	Statistical summary GUI, process database(s)
TC.I06	App setup test	App setup GUI, app setup management, app setup database
TC.I07	Text messaging alert test	User login ID, user ID authentication, Dashboard GUI, Alerts management, app setup database, process database(s)
TC.I08	Auto data acquisition test (CSV files)	Auto data acquisition, data processing, Nelson rules, process database(s)

TABLE 11 Data entry and computation validation

ID	Test step	Result
TC.I01	1. Copy and paste 25 measurement data sets into the “Data” sheet of the Test process spreadsheet.	-
	2. Manually enter another five sets of measurement data via the web browser's Data Entry form.	-
	3. Confirm that all manually entered data is correctly captured in the Form Response Sheet.	Manually entered data is correctly captured in the “Form Response” sheet.
	4. Confirm that all manually entered data has been copied to the “Data” sheet.	Manually entered data are appended to Data sheet.
	5. Confirm that the most recent 25 sets of data are copied from the “Data” sheet to the “Computation” sheet.	The last 25 sets of data are copied to the Computation sheet.
	6. In the test Excel spreadsheet, copy and paste the most recent 25 sets of measurement data from the “Data” sheet.	-
	7. Compare the generated values to the results computed separately.	The computed data in the Computation sheet matches up to the results calculated separately.

TABLE 12 System test cases

ID	Test case description	SR	Actor
TC.S01	Manual data entry	1,3,5,6,7,8	Operator
TC.S02	User sequential GUI operations (Dashboard, Control chart, Statistics summary)	1,3,5,6,7,8,9	Administrator
TC.S03	User sequential GUI operations (Dashboard, Control chart, Statistics summary)	1,3,5,6,7,8,9	Engineer
TC.S04	App parameters setup operation	1,2,3,5,6,8	Administrator
TC.S05	User alerts notification and messaging	1,3,4,5,6,8	Administrator

4.6 | Unit testing phase

The white box testing method is used to examine the functionality of individual modules during unit testing. Fifteen test cases (Table 8) are used for unit testing. An example of the test case is presented in Table 9.

The Nelson rules module necessitates the creation of separate unit tests for each rule. Rules 4, 5, and 6 were challenging to implement because of the relatively complex monitoring conditions they required. The data processing test was designed to identify and eliminate programming errors in equations and cell references.

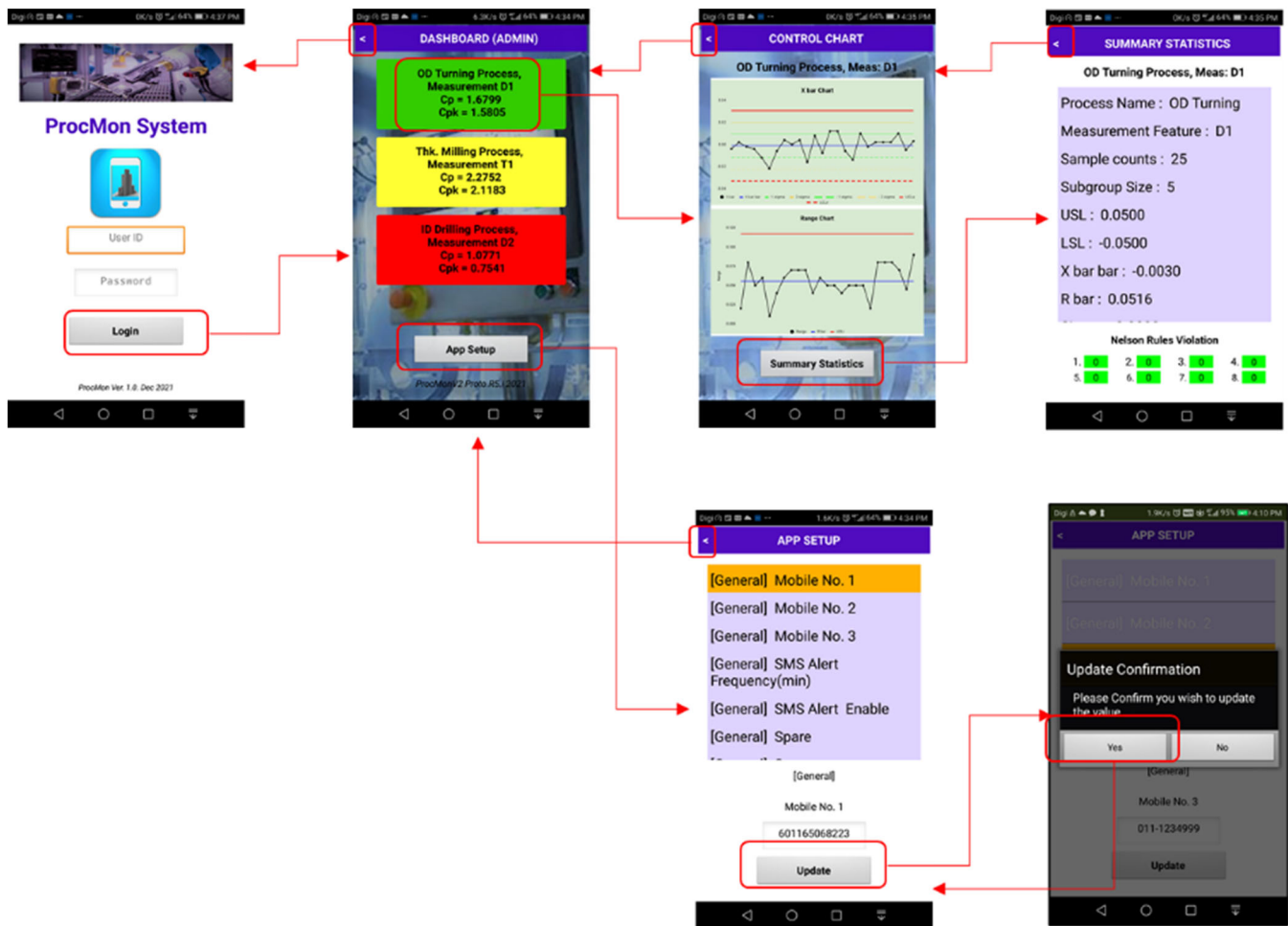


FIGURE 12 Graphical user interfaces (GUIs) developed for app

4.7 | Integration testing phase

The combined functionality of multiple modules is tested using eight test cases in the integration testing phase (Table 10). Table 11 depicts the data entry and computation validation process in Google Drive, involving the interaction between the data entry GUI, data processing, Nelson rules check, and the process database(s).

4.8 | System testing phase

Five test cases (Table 12) are generated based on the app wire flow and intended to mimic real-world use to examine the entire application environment. The wire flow of the GUI is depicted in Figure 12. GUI design must make navigation as simple as possible to comply with DS7, including the appropriate use of color-coded indicators, controls, font sizes, scroll-down lists, and a minimalist design approach. The TC.S04 system test is used to demonstrate an example of a system test (Table 13).

4.9 | Acceptance testing phase

Acceptance testing is carried out with the help of a focus group of 10 participants (subject matter experts) from the manufacturing industry. After receiving an overview of the app's features, they are given the opportunity to install and test the mobile app using predefined use case scenarios on their mobile phones. Following that, the participants complete a questionnaire survey regarding the fulfillment of URs. With the exception of Question 5, at least 90% of participants strongly agree or agree with the question statements. In Question 5, 60% of participants agree or strongly

TABLE 13 System testing for app parameters setup use case

ID	Test step	Result
TC.S05	1. Launch the application and log in with Admin credentials.	Successfully login to the Dashboard GUI.
	2. In the Dashboard GUI, verify that the title of the GUI is "Dashboard (Admin)".	The GUI is titled "Dashboard (Admin)".
	3. Ensure that the app setup button is NOT grayed out.	The app setup button is not grayed out.
	4. Click the app setup button.	Transfer to the app setup GUI.
	5. In the app setup GUI, scroll down the setup parameter list and click on the parameter name to update.	Focus is placed on the parameter name.
	6. Verify the parameter value display in the textbox.	The current value of the parameter is displayed in the text box. The displayed values corresponded to the values stored in the test App configuration spreadsheet.
	7. Modify the parameter value in the text box and then click the Update button. In the confirmation Notification box, click the Yes button.	At the end, the text box display shall be emptied.
	8. Verify that the parameter value in the app setup spreadsheet has been modified.	The app setup spreadsheet has been updated.
	9. Repetition of Steps 5 and 6.	The displayed values correspond to the values stored in the test App configuration spreadsheet (amended values).
	10. Simply click the GUI's title bar.	Switch to the Dashboard GUI.

agree with its statement about the effectiveness of text messaging. The findings (Table 14) therefore show that majority agreed that the app meets the requirements (URs) identified during the requirement analysis phase (Questions 1–9). They also respond positively to the app's interface design (Questions 10–16).

5 | DISCUSSION

The study proposes incorporating FQFD into a V-model app development model. According to Jackson,⁵⁰ the verification phases would have included two critical software design components: conceptual design, which includes domain requirements analysis, project scoping, and specification, and representation design, which includes architecture and module design. FQFD^a is centered on the first critical component to gather requirements and convert them into development objectives.

The requirement analysis phase of the case study solicits VOC through an informed discussion and a user survey questionnaire. Targets in product development are frequently measurable and quantifiable in traditional QFD. In app development, especially when dealing with a new app, various design alternatives and users' lack of technical knowledge complicate requirement solicitation. Given that people's assessments of concepts are always subjective and thus imprecise, FQFD^a uses a 9-point linguistics variable to gauge user preference on nine proposed features. The survey responses are converted into fuzzy numbers for the relative importance rating analysis.

The second critical component is addressed by aligning URs, SRs, and DSs throughout the design process to reduce the possibility of not meeting VOC. Such alignment helps the team members to use a reasonable approach to structure the design challenge. During the system design phase of the case study, FQFD^b guides the design team in identifying and rating the SRs based on 5-point linguistic variables in this round to create a distinct boundary that would influence the design team to decide their preference. The relative importance ratings reveal that the top three SRs contribute 41.21% of the total importance. Therefore, synchronized data communications, dynamic data linkage, and a user-friendly and easy-to-navigate GUI are more critical to the customer than the other listed requirements. Either 9- or 5-point linguistic variables allow survey participants and the design team to define their choices in an understandable language easily. Fuzzy number computations are more complex than crisp number computations. The former captures value ranges and avoids information loss, making it more representative than the latter, resulting in a single average value after computation. The HOQ^b roof emphasizes the importance of using an open-source programming platform (SR1) to enable SR2, SR3, SR5, and SR7. The design team considers DSs in architecture design. HOQ^b directs the team to concentrate on the top three DSs.

The resultant architecture design is a solution using mobile cloud computing technology to provide interoperability between different client devices and access to synchronized information at any place and time, as highlighted by these DSs. The system is a client–server architecture with a three-tier design that includes the Google Drive cloud service, Android mobile devices, onsite data entry stations, and a mobile app. The HOQ

TABLE 14 Summary of acceptance test questionnaires results

ID	Survey question	UR ref.	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
1	This app is a low-cost solution for monitoring manufacturing processes in real time.	UR2	40%	60%	0%	0%	0%
2	This app is flexible and easy to customize.	UR8	30%	70%	0%	0%	0%
3	This app provides 24/7 and “on-the-go” access to information about factory process monitoring.	UR1	70%	30%	0%	0%	0%
4	This app provides adequate data security through the use of a password-protected login.	UR7	60%	30%	10%	0%	0%
5	For process monitoring, text messaging is an effective user alert feature.	UR6	10%	50%	30%	10%	0%
6	The app's GUI layout is neat and clear.	UR9	30%	60%	10%	0%	0%
7	By allowing for the computation of process capability via the DNOM method, this app is well-suited for high-mix, low-volume manufacturing (short production runs).	UR3	50%	50%	0%	0%	0%
8	This app simplifies the process of computation, data analysis, and control chart generation through the use of automated processes.	UR4	50%	50%	0%	0%	0%
9	This app is an efficient tool for monitoring statistical processes in the factory.	UR5	40%	50%	10%	0%	0%
10	The app operates in a responsive manner.	UR9	60%	40%	0%	0%	0%
11	The data or screen is updated on a regular basis.	UR9	20%	80%	0%	0%	0%
12	The GUI is well-designed and easy to navigate.	UR9	40%	50%	10%	0%	0%
13	The GUI design theme is appropriate and comfortable to view.	UR9	40%	50%	10%	0%	0%
14	The GUI's content and text are easily readable.	UR9	40%	60%	0%	0%	0%
15	The language used in the graphical user interface is simple to understand.	UR9	50%	50%	0%	0%	0%
16	The GUI buttons and their layout are intuitive.	UR9	50%	50%	0%	0%	0%

roof investigates the correlations between DSs and assists in allocating higher priority and resources to those with high correlations to other DSs. The case prioritizes DS3 because identifying open-source programming and cloud computing platforms is critical for programming-related DSs (DS4, DS5, DS8, DS9, and DS10). Application modules of various architecture layers are defined in the module design to fulfill the tasks determined in the architectural design. During the coding phase, the modules are realized using open-source tools.

Incorporating FQFDs into the V-model yields the following benefits. First and foremost, the focus and breakdown of customer requirements are more organized. Decision-making is facilitated based on consensus when all information is presented in HOQ. Second, significant issues and problems can be identified during the planning (verification) stage, enabling early measurement and thus reducing app development time. Third, explicit resource prioritization and allocation, including the design of test plans, can be achieved. However, prioritization does not always correspond to development efforts because some priorities may benefit from the availability of required technology. Compared with Han et al.⁶¹'s V-model, the FQFD in the verification stages that have proven does not required to deliberate the trade-off relationships between URs, and therefore, the relevant elicitation process is more straightforward. This attribute allows the proposed V-model to reach faster decision on UR and SR as well as DSs and hence is more suitable for small mobile app with predefined application scope and user base.

Only two stages of FQFD are deployed in the model and case study, spanning three phases of the model. Such an arrangement corresponds to the standard path of app development,³³ such as URs' system design and architecture design. FQFD is no longer used in module design because it involves a fine breakdown of technical specifications, which frequently includes direct judgment. With the increasing complexity of mobile apps, the V-model can easily be improved, for example, by adding FQFD to allow for more levels of decomposition before reaching low-level programming when the need arises. Therefore, the modified model is adaptable to the majority of mobile apps. Furthermore, the VOC is made up entirely of end users. The primary benefit is the development of a highly focused app. In a broader sense, customers can refer to various stakeholders, including the developer and the sponsor. The clear implication (especially for FQFD^a) is that the requirements of these stakeholders must be collected, and the importance of these requirements must be assessed separately in their corresponding groups before being normalized. Furthermore, stakeholders with competing interests may complicate the deliberation of DSs.

The validation phases use the test cases designed in the verification phases to detect and correct programming errors. Unit testing debugs the primary module coding and increases productivity by avoiding time-consuming reworks in the future. Integration testing validates more complex functions that combine different modules. The test cases are instrumental in identifying any connectivity or linkage issues between modules. System testing determines whether the mobile app can pass all use case testing at the system level. It provides the developer with a better understanding and feels for the UX. Finally, a focus group completes a survey questionnaire and conducts a user acceptance test via the app's trial run. Overall, the focus group consistently agrees that the mobile app met all the requested URs.

The completion of all validation phases thus concludes that the design activities are successful. The concurrent development and refinement of design specifications and tests are credited with success. The practice allows for the timely breakdown of tasks and the placement of attention to all elements of the newly generated layer of task breakdown one at a time. Furthermore, whenever necessary, references to previously developed design specifications and tests can be sought (e.g., a HOW can be traced back to the corresponding WHAT). The model makes the decision-making process and information flow explicit; for example, HOQ and test plans are "living documents." Given that studying the app and its subsystems simultaneously is possible, the resulting comprehensibility aids in the system's design. As a result, test plan fulfillment becomes goal-oriented. Any errors or deficiencies in the system can be traced back to specific subsystems, limiting the scope of detailed error searching.

6 | CONCLUSION

Mobile app development is challenging because of rapidly changing business requirements and technical constraints for mobile systems. This study proposed a V-model integrating with two FQFDs for app development. The fuzzy set theory in FQFD aids to acquire more precise deductions of human (user or design team) inputs affected by intrinsic vagueness and ambiguity in linguistics. FQFDs were deployed in the verification phases to gather and decompose URs into app specifications for architecture and module designs. Test plans were thought-out in the verification phases. On the basis of these test plans, the validation phases assembled and tested subsystems of codes. The model was demonstrated through an online SPC app development. The results of the acceptance test survey revealed that the app routinely meets URs, demonstrating the model's capacity to connect mobile app development with the VOC. As the research implications, the work improves the user-centered design of V-model through integrating FQFDs for efficient UR elicitation. This is essential given that URs are often vague and misunderstood, resulting in poor mobile app design. As the managerial implications, with largely common steps, the model can be generic and adaptable to other development-based projects. The structure supports practitioners to systematically develop mobile apps; for instance, the outputs of FQFD can be developed naturally into test designs. On the limitations, the development has just reached the proof-of-concept stage, and the model does not account the full life cycle of mobile apps, such as software managerial aspects, and maintenance. Future research directions include the incorporation of HOW measurables, the effective prioritizing of HOWs in the action plan, and the flexible V-model phases through incorporation of agile concept.

CONFLICT OF INTEREST

There is no conflict of interest in the research.

ETHICS APPROVAL STATEMENT

Not applicable.

PATIENT CONSENT STATEMENT

Not applicable.

PERMISSION TO REPRODUCE MATERIAL FROM OTHER SOURCES

There is no reproduction of material from other sources.

CLINICAL TRIAL REGISTRATION

Not applicable.

DATA AVAILABILITY STATEMENT

There is no data published openly.

ORCID

Chong Hon Lim  <https://orcid.org/0000-0002-6789-7735>

Jeng Feng Chin  <https://orcid.org/0000-0002-9547-3673>

REFERENCES

1. Aldayel A, Alnafjan K. Challenges and best practices for mobile application development: review paper. In Proceedings of the International Conference on Compute and Data Analysis (ICCCA '17). *Assoc Comput Mach.* 2017;41-48. doi:[10.1145/3093241.3093245](https://doi.org/10.1145/3093241.3093245)
2. Werth O, Guhr N, Bretnier M. Successful mobile application development: towards a taxonomy of domain-specific process models and methodologies. In *Proceedings of the 52nd Hawaii International Conference on System Sciences*; 2019:7461-7470.
3. Al-Ratrout S, Tarawneh O, Altarawneh MH, Altarawneh MY. Mobile application development methodologies adopted in Omani market: a comparative study. *Int J of Softw Eng & Appl.* 2019;10(2):13-22. <https://ssrn.com/abstract=3376391>
4. Kaur A, Kaur K. Systematic literature review of mobile application development and testing effort estimation. *J King Saud University - Comput Inf Sci.* 2018;34(2):1-15. doi:[10.1016/j.jksuci.2018.11.002](https://doi.org/10.1016/j.jksuci.2018.11.002)
5. Thiagarajan, C, 2020, A systematic study on agile software development methodologies and practices. *JETIR* 2020; 7(3):575-581.
6. Han Y, Lee D, Choi B, Hinchey M, In HP. Value-driven V-model: from requirements analysis to acceptance testing. *IEICE Trans Inf Syst.* 2016;E99.D(7):1776-1785. doi:[10.1587/transinf.2015EDP7451](https://doi.org/10.1587/transinf.2015EDP7451)
7. Graessler I, Hentze J, Bruckmann T. V-models for interdisciplinary systems engineering. *International Design Conference - Design* 2018; 2018:747-756.
8. Jain V, Harish R, Khan SA, Ali S, Rajat RN. Software process and models—a brief study. *Discovery Eng.* 2013;2(7):24-29.
9. Rastogi V. Software development life cycle models—comparison, consequences. *Int J Comput Sci Inf Technol.* 2015;6(1):168-172.
10. Grinter RE. Recomposition: putting it all back together again. In Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work. *Association for Computing Machinery*; 1998:393-402.
11. Hao L, Zhang J, Ma X. Development trend of software process model. *IOP Conf Series: Mater Sci Eng.* 2019;569(5):1-6. doi:[10.1088/1757-899X/569/5/052077](https://doi.org/10.1088/1757-899X/569/5/052077)
12. Khan ME, Shadab S, Khan F. Empirical study of software development life cycle and its various models. *Int J Softw Eng.* 2020;8(2):16-26.
13. Jeong J, Heo G. Cyber security evaluation for nuclear I&C systems corresponding to V-model. In Transactions of the Korean Nuclear Society Virtual Spring Meeting July 9-10, 2020.
14. Essebaa I, Chantit S. A combination of V development life cycle and model-based testing to deal with software system evolution issues. In Proceedings of the 6th International Conference on Model-Driven Engineering and Software Development (MODELSWARD 2018); 2018:528-535. doi:[10.5220/0006657805280535](https://doi.org/10.5220/0006657805280535)
15. Lu G, Lu S, Li X. Control algorithm development for electric vehicle residual driving distance based on V model. In: *International Conference on Modeling, Simulation, Optimization and Numerical Techniques*. Atlantis Press; 2019:175-178.
16. Nuankaew P, Phanniphong K, Nasa-ngium P, Nuankaew W. Public transport knowledge management in local communities. *Turk J Comput Math Educ.* 2021;12(6):2204-2218.
17. Rachmaniah M, Suroso A, Syukur M, Hermadi I, Putra RW, Tirani MRG. Smart campus attempt using green transportation mobile-based tracking app. *IOP Conf Ser Mater Sci Eng.* 2020;874(1):1-9. doi:[10.1088/1757-899X/874/1/012020](https://doi.org/10.1088/1757-899X/874/1/012020)
18. Dalimunthe AH, Sari DS, Kurniasari P. The design of online promotion system for entrepreneur in Palembang City based on mobile application. In Proceedings of the Sriwijaya International Conference on Information Technology and Its Applications (SICONIAN 2019); 2019:73-80.
19. Mathur S, Malik S. Advancements in the V-model. *Int J Comput Appl.* 2010;1(12):29-34. doi:[10.5120/266-425](https://doi.org/10.5120/266-425)
20. Durmus M, Üstoglu I, Tsarev R, Börcsök J. Enhanced V-model. *Informatica.* 2018;42(4):577-585. doi:[10.31449/inf.v42i4.2027](https://doi.org/10.31449/inf.v42i4.2027)
21. Graessler I. A new V-model for interdisciplinary product engineering. 59th Ilmenau Scientific Colloquium. 2017.
22. Dasanayake S, Aaramaa S, Markkula J, Oivo M. Impact of requirements volatility on software architecture: how do software teams keep up with ever-changing requirements? *J Softw: Evol Process.* 2019;31(6):1-21. doi:[10.1002/smr.2160](https://doi.org/10.1002/smr.2160)
23. Lu C, Lin L, Yeh H. A multi-phased FQFD for the design of brand revitalization. *Total Qual Manag.* 2019;30(8):848-871. doi:[10.1080/14783363.2017.1343139](https://doi.org/10.1080/14783363.2017.1343139)
24. Punz S, Hehenberger P, Follmer M, Zeman K. Customer oriented concept development in mechatronic product design. In Proceedings of the International Design Conference - DESIGN 2010; 2010:927-936.
25. Cui F, Ma L, Hou G, Pang Z, Hou Y, Li L. Development of smart nursing homes using systems engineering methodologies in industry 4.0. *Enterp Inf Syst.* 2020;14(4):463-479. doi:[10.1080/17517575.2018.1536929](https://doi.org/10.1080/17517575.2018.1536929)
26. Doyle F, Cosgrove J. Steps towards digitization of manufacturing in an SME environment. *Procedia Manuf.* 2019;38:540-547. doi:[10.1016/j.promfg.2020.01.068](https://doi.org/10.1016/j.promfg.2020.01.068)
27. Baldauf M, Müller S, Seeliger A, Küng T, Michel A, Züllig W. Human interventions in the smart factory—a case study on co-designing mobile and wearable monitoring systems with manufacturing staff. In Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems. Association for Computing Machinery, Article 470; 2021:1-6. doi:[10.1145/3411763.3451774](https://doi.org/10.1145/3411763.3451774)
28. Patil V, Phafat N, Dolas D. Quality function deployment: case study on roadmap for impeller design. *Int J Ind Eng.* 2018;11(1):25-29. doi:[10.26488/IEJ.11.1.1034](https://doi.org/10.26488/IEJ.11.1.1034)
29. Hauser J, Griffin A, Klein R, Katz G, Gaskin S. Quality function deployment. *Wiley International Encyclopaedia of Marketing* 2010:1-16. doi:[10.1002/9781444316568.wiem05023](https://doi.org/10.1002/9781444316568.wiem05023)
30. Jamkhaneh HB, Shahin R, Leva MC. A new framework for optimization and effectiveness of service operations approaches to reduce human errors in healthcare. *Int J Hum Factors Ergon.* 2021;8(2):157-182. doi:[10.1504/IJHFE.2021.116067](https://doi.org/10.1504/IJHFE.2021.116067)
31. Chan LK, Wu ML. Quality function deployment: a comprehensive review of its concepts and methods. *Qual Eng.* 2002;15(1):23-35. doi:[10.1081/QEN-120006708](https://doi.org/10.1081/QEN-120006708)
32. Erdil NO, Arani OM. Quality function deployment: more than a design tool. *Int J Qual Serv.* 2019;11(2):142-166. doi:[10.1108/IJQSS-02-2018-0008](https://doi.org/10.1108/IJQSS-02-2018-0008)
33. Jamkhaneh HB, Shahin R, Shahin A, ArabYarmohammadi M. CMMS software quality function deployment based on maintenance objectives: a framework for software selection process. *Int J Product Qual.* 2021;32(4):413-439. doi:[10.1504/IJPQM.2021.114260](https://doi.org/10.1504/IJPQM.2021.114260)
34. Shen X, Tan K, Xie M. The implementation of quality function deployment based on linguistic data. *J Intell Manuf.* 2000;12(1):65-75. doi:[10.1023/A:1008955630880](https://doi.org/10.1023/A:1008955630880)
35. Karsak EE. Fuzzy multiple objective decision making approach to prioritize design requirements in quality function deployment. *Int J Prod Res.* 2004;42(18):3957-3974. doi:[10.1080/00207540410001703998](https://doi.org/10.1080/00207540410001703998)

36. Chan LK, Wu ML. A systematic approach to quality function deployment with a full illustrative example. *Omega*. 2005;33(2):119-139. doi:[10.1016/j.omega.2004.03.010](https://doi.org/10.1016/j.omega.2004.03.010)
37. Zahedi M, Yousefi S, Cheshmbera M. A fuzzy quality function deployment approach to enterprise resource planning software selection. *Asian J Sci Res*. 2011;4(2):114-128. doi:[10.3923/ajsr.2011.114.128](https://doi.org/10.3923/ajsr.2011.114.128)
38. Jia GZ, Bai M. An approach for manufacturing strategy development based on fuzzy-QFD. *Comput Ind Eng*. 2011;60(3):445-454. doi:[10.1016/j.cie.2010.07.003](https://doi.org/10.1016/j.cie.2010.07.003)
39. Gani AN, Mohamed Assarudeen SN. A new operation on triangular fuzzy number for solving fuzzy linear programming problem. *Appl Math Sci*. 2012;6(11):525-532.
40. Miller GA. The magical number seven, plus or minus two: some limits on our capacity for processing information. *Psychol Rev*. 1956;63(2):81-97. doi:[10.1037/h0043158](https://doi.org/10.1037/h0043158)
41. Taherdoost H. What is the best response scale for survey and questionnaire design; review of different lengths of rating scale/attitude scale/Likert scale. *Int J Acad Res Manag*. 2019;8(1):1-10.
42. Preston C, Colman A. Optimal number of response categories in rating scales: reliability, validity, discriminating power, and respondent preferences. *Acta Psychol (Amst)*. 2000;104(1):1-15. doi:[10.1016/S0001-6918\(99\)00050-5](https://doi.org/10.1016/S0001-6918(99)00050-5)
43. Facchinetti G, Ricci RG, Muzzioli S. Note on ranking fuzzy triangular numbers. *J Intell Syst*. 1998;13(7):613-622. doi:[10.1002/\(SICI\)1098-111X\(199807\)13:7%3C613::AID-INT2%3E3.0.CO;2-N](https://doi.org/10.1002/(SICI)1098-111X(199807)13:7%3C613::AID-INT2%3E3.0.CO;2-N)
44. Kang K, Cohen S, Hess J, Novak W, Peterson A. *Feature-Oriented Domain Analysis (FODA) Feasibility Study*. Pittsburgh, Pennsylvania: Software Engineering Institute, Carnegie Mellon University, Technical Report CMU/SEI-90-TR-021.; 1990.
45. Apel S, Kästner C. An overview of feature-oriented software development. *J Object Technol*. 2009;8(5):49-84. doi:[10.5381/jot.2009.8.5.c5](https://doi.org/10.5381/jot.2009.8.5.c5)
46. Pyzdek T, Keller P. *Six Sigma Handbook: A Complete Guide for Green Belts, Black Belts, and Managers at All Levels*. 3rd ed. McGraw-Hill; 2010.
47. Montgomery D. *Introduction to Statistical Quality Control*. 7th ed. John Wiley and Sons; 2013.
48. Liu C, Yang W, Li Z, Yu Y. Recommending software features to designers: from the perspective of users. *Softw - Pract Exp*. 2020;50(9):1778-1792. doi:[10.1002/spe.2845](https://doi.org/10.1002/spe.2845)
49. Capilla R, Kazman R, Romera C, Carrillo C. Usability implications in software architecture: the case study of a mobile app. *Softw: Pract Exper*. 2020;50(12):2145-2168. doi:[10.1002/spe.2883](https://doi.org/10.1002/spe.2883)
50. Jackson D. Towards a theory of conceptual design for software. In *ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. Association for Computing Machinery; 2015:282-296.

How to cite this article: Lim CH, Chin JF. V-model with fuzzy quality function deployments for mobile application development. *J Softw Evol Proc*. 2023;35(1):e2518. doi:[10.1002/smr.2518](https://doi.org/10.1002/smr.2518)